

When Postgres Misbehaves

Locks, Monitoring, Key Config & Reading Query Plans

Session 3 of 3 — Beginning Postgres Workshop

PGSummit NYC 2026

Session 3 Topics

- Locks & blocking, with a live demo
- Finding and stopping bad queries, timeouts as guardrails
- Monitoring essentials and `pg_stat_statements`
- A few key config settings (`shared_buffers`, `work_mem`)
- Reading query plans with EXPLAIN
- Index basics and common performance patterns

Locks, Blocking & Runaway Queries

Common Problems

- Long-running queries blocking others
- Idle transactions holding locks
- Runaway queries consuming resources
- Connection exhaustion
- Lock contention

Postgres Locks

ACCESS SHARE	SELECT COPY TO	Allows Reads	Allows writes
ROW SHARE	SELECT ... FOR UPDATE SELECT ... FOR SHARE	Allows Reads	Allows writes
ROW EXCLUSIVE	INSERT UPDATE COPY FROM TRUNCATE DELETE	Allows Reads	Allows writes
SHARE UPDATE EXCLUSIVE	VACUUM CREATE INDEX CONCURRENTLY	Allows Reads	Blocks writes
SHARE	CREATE INDEX ALTER TABLE ... ADD FOREIGN KEY	Allows Reads	Blocks writes
SHARE ROW EXCLUSIVE	CREATE TABLE ... AS SELECT	Allows Reads	Blocks writes
EXCLUSIVE	REFRESH MATERIALIZED VIEW	Allows Reads	Blocks writes
ACCESS EXCLUSIVE	ALTER TABLE (most operations) DROP TABLE REINDEX TRUNCATE LOCK TABLE	Blocks Reads	Blocks writes



Demo: Create a Blocking Lock

Window 1

```
BEGIN;  
UPDATE bluebox.film  
SET popularity = popularity  
WHERE film_id = 155;  
-- no COMMIT yet: this session is  
-- now "idle in transaction"
```

Window 2

```
UPDATE bluebox.film  
SET popularity = popularity  
WHERE film_id = 155;  
-- hangs: waiting for window 1's  
-- row lock
```

Find the blocker from a third window (next slide). ROLLBACK in window 1 and window 2 finishes instantly.

Who Is Blocking Whom?

```
SELECT pid,  
       pg_blocking_pids(pid) AS blocked_by,  
       wait_event_type,  
       NOW() - query_start AS waiting,  
       LEFT(query, 40) AS query  
FROM pg_stat_activity  
WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

pid	blocked_by	wait_event_type	waiting	query
166	{164}	Lock	00:00:02.104775	UPDATE bluebox.film SET popularity = pop

Look up pid 164 in pg_stat_activity: it's window 1, idle in transaction.
That pid is what we cancel or terminate in a moment.

Finding Long-Running Queries

```
SELECT
    pid,
    NOW() - query_start as duration,
    username,
    state,
    query
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start
LIMIT 10;
```

Finding Idle Transactions

-- Idle transactions can hold locks!

```
SELECT
    pid,
    NOW() - xact_start as transaction_duration,
    NOW() - state_change as idle_duration,
    username,
    query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
    AND xact_start IS NOT NULL
ORDER BY xact_start;
```

Canceling a Query

```
-- Cancel the current query (graceful)  
SELECT pg_cancel_backend(12345);
```

```
-- Returns true if signal sent successfully
```

The query receives an interrupt and can clean up

Terminating a Connection

-- Terminate the entire connection (forceful)

```
SELECT pg_terminate_backend(12345);
```

-- Terminate all connections to a database except our own

```
SELECT pg_terminate_backend(pid)
```

```
FROM pg_stat_activity
```

```
WHERE datname = 'bluebox'
```

```
    AND pid != pg_backend_pid();
```

Statement Timeout

Automatically kill queries that run too long

-- Set for session

```
SET statement_timeout = '30s';
```

-- Set for a single transaction

```
BEGIN;
```

```
SET LOCAL statement_timeout = '10s';
```

```
SELECT * FROM huge_table; -- Will timeout after 10s
```

```
COMMIT;
```

-- Set per user

```
ALTER ROLE app_user SET statement_timeout = '60s';
```

Idle Transaction Timeout

Kill sessions that sit idle in a transaction

-- postgresql.conf or per-session

idle_in_transaction_session_timeout = '10min'

-- Per user

ALTER ROLE app_user SET idle_in_transaction_session_timeout = '5min';

Lock Timeout

Don't wait forever for locks

-- Set lock timeout

```
SET lock_timeout = '5s';
```

-- Now this will fail fast if table is locked

```
ALTER TABLE bluebox.rental ADD COLUMN new_col INT;
```

-- ERROR: canceling statement due to lock timeout

Monitor Postgres

- Catch problems before users notice
- Capacity planning
- Performance baselines
- Audit trails
- Sleep better at night

What to Monitor

System Level

- CPU usage
- Memory usage
- Disk I/O
- Network
- Disk space

Postgres Level

- Connections
- Transaction rate
- Cache hit ratio
- Replication lag
- Locks

Key Metrics to Watch

```
-- Connection count
SELECT count(*) FROM pg_stat_activity;

-- Database size growth
SELECT pg_database_size('bluebox');

-- Transaction rate (per second)
SELECT (xact_commit + xact_rollback) / extract(epoch from now() - coalesce(stats_reset,
(pg_catalog.pg_stat_file('base/' || datid || '/PG_VERSION')).modification)) AS tps
FROM pg_stat_database
WHERE datname = 'bluebox';
```

Cache Hit Ratio

```
SELECT
    datname,
    ROUND(
        blks_hit::numeric / NULLIF(blks_hit + blks_read, 0) * 100,
        2
    ) as cache_hit_ratio
FROM pg_stat_database
WHERE datname = 'bluebox';
```

-- Should be > 99% for OLTP workloads

Monitoring Tools

Open Source

pg_stat_monitor

Prometheus + postgres_exporter

Grafana

pgwatch

pgmonitor

Commercial

pganalyze

Datadog

New Relic

Simple Health Check Query

```
SELECT
    'connections' as metric,
    count(*)::text as value
FROM pg_stat_activity
UNION ALL
SELECT
    'database_size',
    pg_size_pretty(pg_database_size('bluebox'))
UNION ALL
SELECT
    'active_queries',
    count(*)::text
FROM pg_stat_activity
WHERE state = 'active'
UNION ALL
SELECT
    'oldest_transaction',
    COALESCE(max(NOW() - xact_start)::text, 'none')
FROM pg_stat_activity;
```

pg_stat_statements: Finding Slow Queries

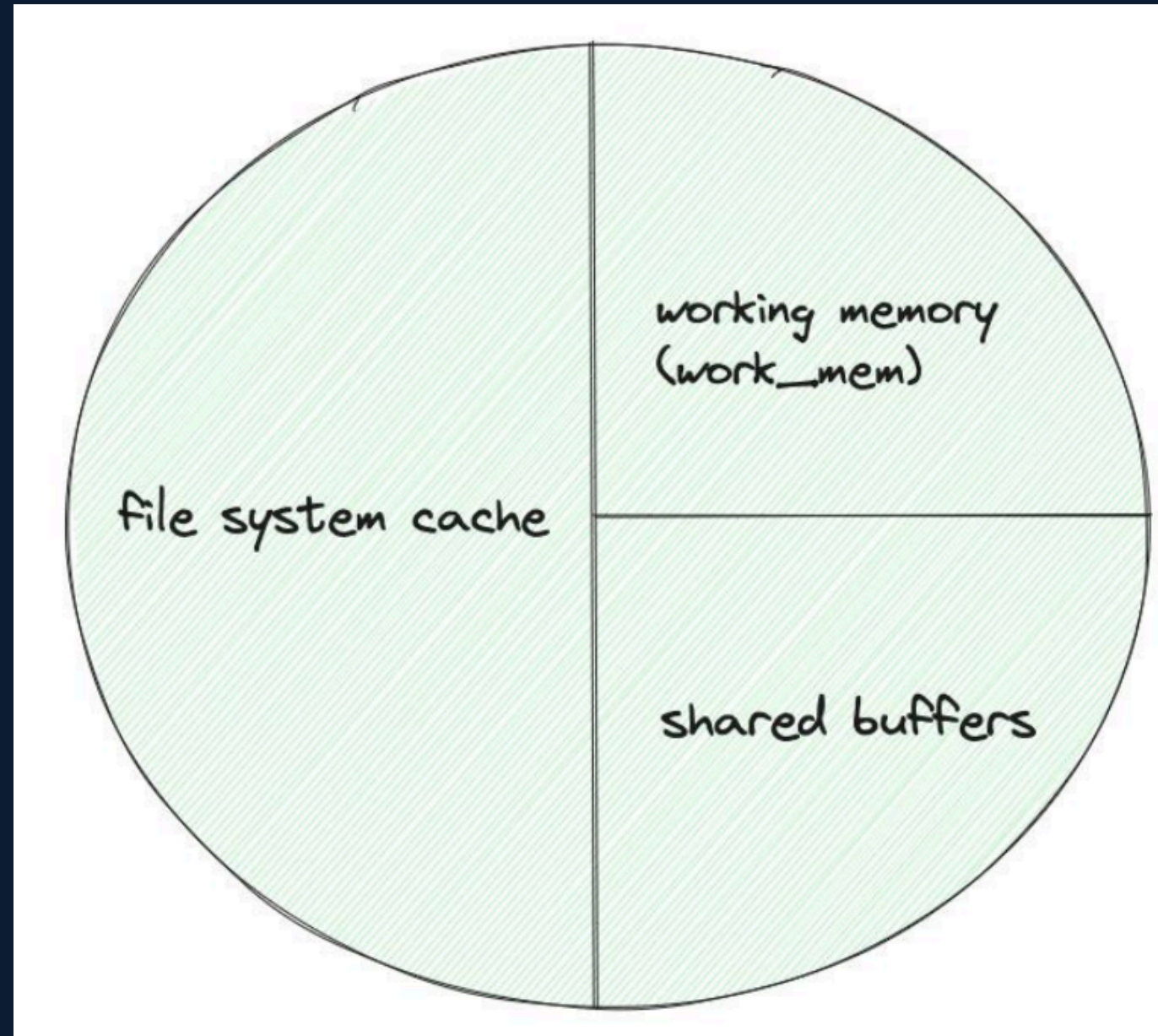
Tracks every query shape: calls, total and average time. It needs `shared_preload_libraries` (our docker-compose.yml preloads it; on your own server, set it and restart).

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

```
SELECT
    LEFT(query, 60) as query,
    calls,
    ROUND(total_exec_time::numeric, 2) as total_ms,
    ROUND(mean_exec_time::numeric, 2) as avg_ms,
    ROUND((100 * total_exec_time /
           SUM(total_exec_time) OVER ()):numeric, 2) as pct
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

The top of this list is what you EXPLAIN - right after a few memory settings.

Memory Configuration



Shared Buffers

The database page cache - most important setting

-- Check current value

`SHOW shared_buffers;`

-- Set in `postgresql.conf`

`shared_buffers = 8GB` -- For 32GB RAM system

-- Requires restart to change

Rule of thumb: Start with 25% of available RAM

Shared Buffers Guidelines

System RAM	shared_buffers
4GB	1GB
16GB	4GB
64GB	16GB
256GB	32-64GB

Beyond 32GB, diminishing returns - OS cache helps too

What is work_mem?

Memory for query operations:

- Sorting (ORDER BY)
- Hash joins
- Hash aggregations
- Window functions

```
SHOW work_mem;  -- Default: 4MB
```

Setting work_mem

Be careful: $\text{work_mem} \times \text{connections} \times \text{operations}$

-- Global setting (conservative)

`work_mem = 64MB`

-- Increase for specific session

`SET work_mem = '256MB';`

-- Run complex analytical query

`RESET work_mem;`

Maintenance Work Memory

Used by maintenance operations:

```
maintenance_work_mem = 1GB
```

```
-- Affects:  
-- VACUUM  
-- CREATE INDEX  
-- ALTER TABLE ADD FOREIGN KEY
```

Can be set much higher than work_mem

When to Increase work_mem

Signs you need more:

-- Check for disk sorts

```
EXPLAIN (ANALYZE, BUFFERS) SELECT ...
```

-- Look for:

-- Sort Method: external merge ← disk sort, increase work_mem

-- Sort Method: quicksort ← memory sort, good!

EXPLAIN - The Essential Tool

What is EXPLAIN?

Shows you the query execution plan

```
EXPLAIN SELECT * FROM bluebox.film WHERE vote_average > 8;
```

Output:

QUERY PLAN

```
Seq Scan on film (cost=0.00..941.95 rows=110 width=777)
```

```
  Filter: (vote_average > '8'::double precision)
```

EXPLAIN Options

-- Basic plan (estimated only)

```
EXPLAIN SELECT * FROM bluebox.film WHERE vote_average > 8;
```

-- With actual execution times

```
EXPLAIN ANALYZE SELECT * FROM bluebox.film WHERE vote_average > 8;
```

-- With buffer/I/O statistics

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM bluebox.film WHERE vote_average > 8;
```

-- All the details in text format

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
```

```
SELECT * FROM bluebox.film WHERE vote_average > 8;
```

Reading Plan Costs

```
Seq Scan on film (cost=0.00..941.95 rows=110 width=777)
      ^^^^          ^^^^^^      ^^^^^^^^^^      ^^^^^^^^^^
           startup    total    estimated    row width
           cost      cost      rows         in bytes
```

- **Cost:** Arbitrary units, relative comparison
- **Rows:** Estimated row count from table statistics (110 films with `vote_average > 8`; ANALYZE and autovacuum keep these fresh)
- **Width:** Average row size in bytes (777 bytes per film row)

EXPLAIN ANALYZE

Shows **actual** execution:

EXPLAIN ANALYZE

```
SELECT * FROM bluebox.film WHERE vote_average > 8;
```

Estimated 110 rows, got 111 —
pretty close!

EXPLAIN ANALYZE, BUFFERS example

type of scan

Seq Scan on weather (cost=0.00..9204.64 rows=3153 width=850)
(actual time=0.010..92.501 rows=3182 loops=1)
Filter: (event_type = 'Winter Storm':text)
Rows Removed by Filter: 75909
Buffers: shared hit=1532 read=6684 ← buffers shared or read from disk
I/O Timings: shared/local read=58.487
Planning Time: 0.066 ms
Execution Time: 92.735 ms ← time spent reading from disk
(7 rows)

Warning About EXPLAIN ANALYZE

⚠ **EXPLAIN ANALYZE actually runs the query!**

-- This will DELETE your data!

```
EXPLAIN ANALYZE DELETE FROM bluebox.customer;
```

-- Use ROLLBACK for data-modifying queries

```
BEGIN;
```

```
EXPLAIN ANALYZE DELETE FROM bluebox.customer;
```

```
ROLLBACK;
```

EXPLAIN with BUFFERS

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM bluebox.film WHERE vote_average > 8;
```

```
Seq Scan on film (cost=0.00..941.95 rows=110 width=777)  
      (actual time=0.034..5.966 rows=111 loops=1)
```

```
    Filter: (vote_average > '8'::double precision)
```

```
    Rows Removed by Filter: 7725
```

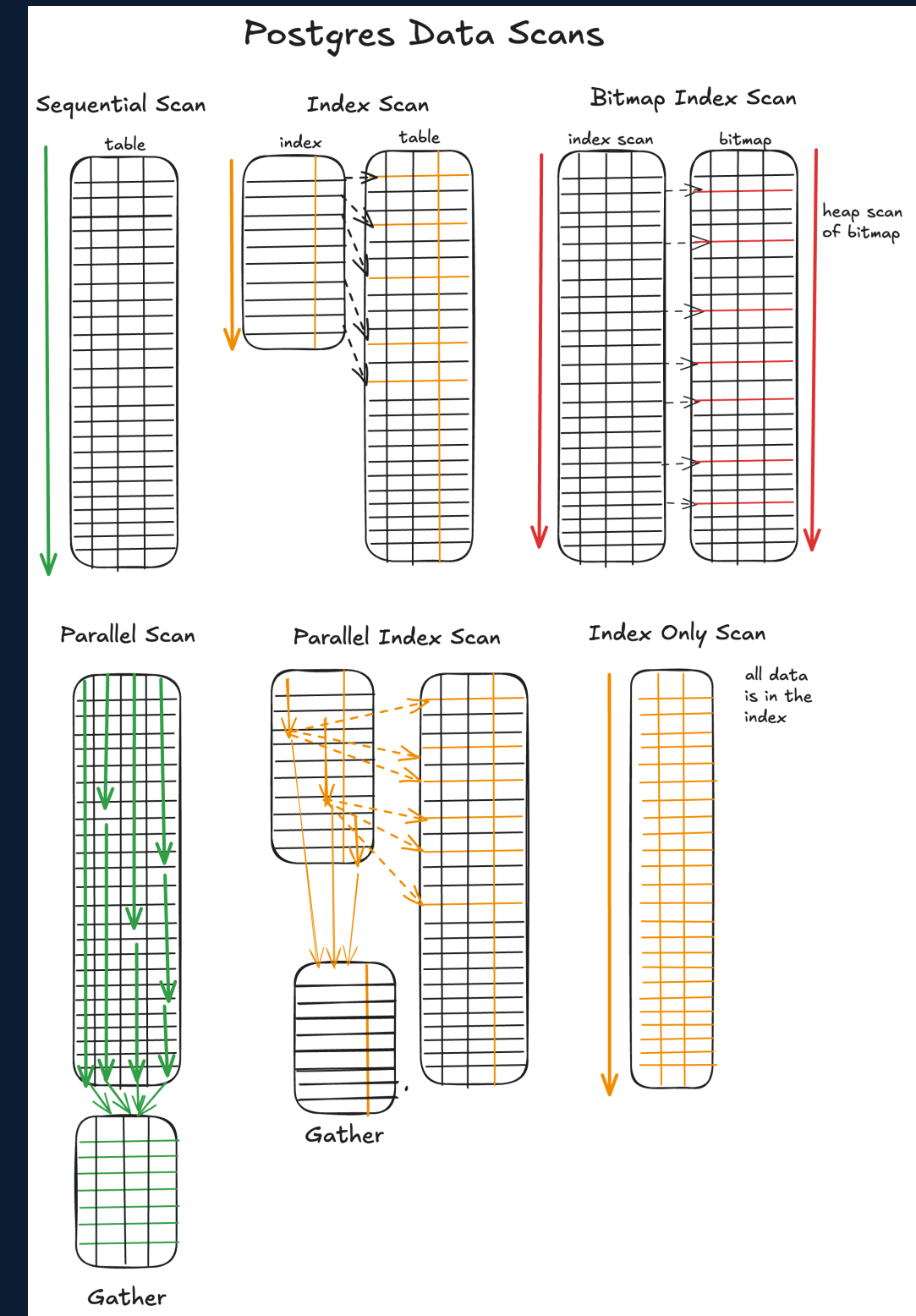
```
    Buffers: shared hit=844
```

```
Planning Time: 1.264 ms
```

```
Execution Time: 6.009 ms
```

- **shared hit=844**: 844 pages found in cache ✓
- **shared read**: Pages read from disk (none here!)

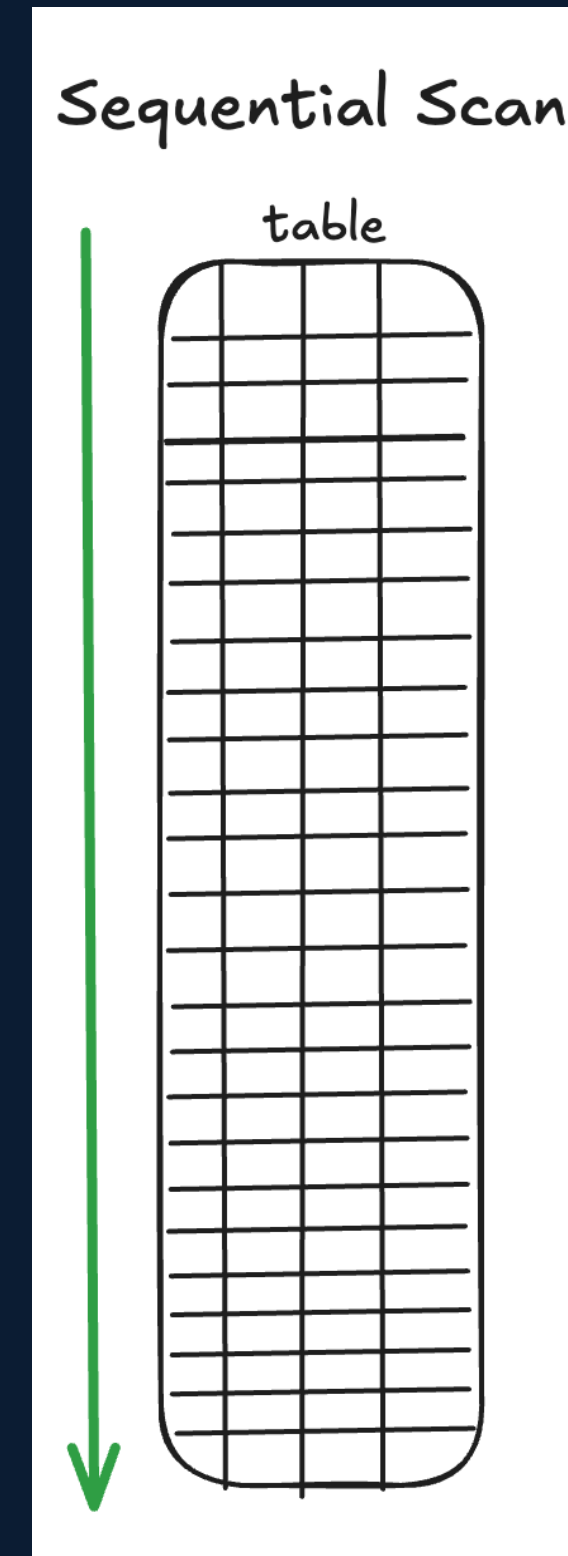
Common Plan Operations



Sequential Scan

Reads every row in the table

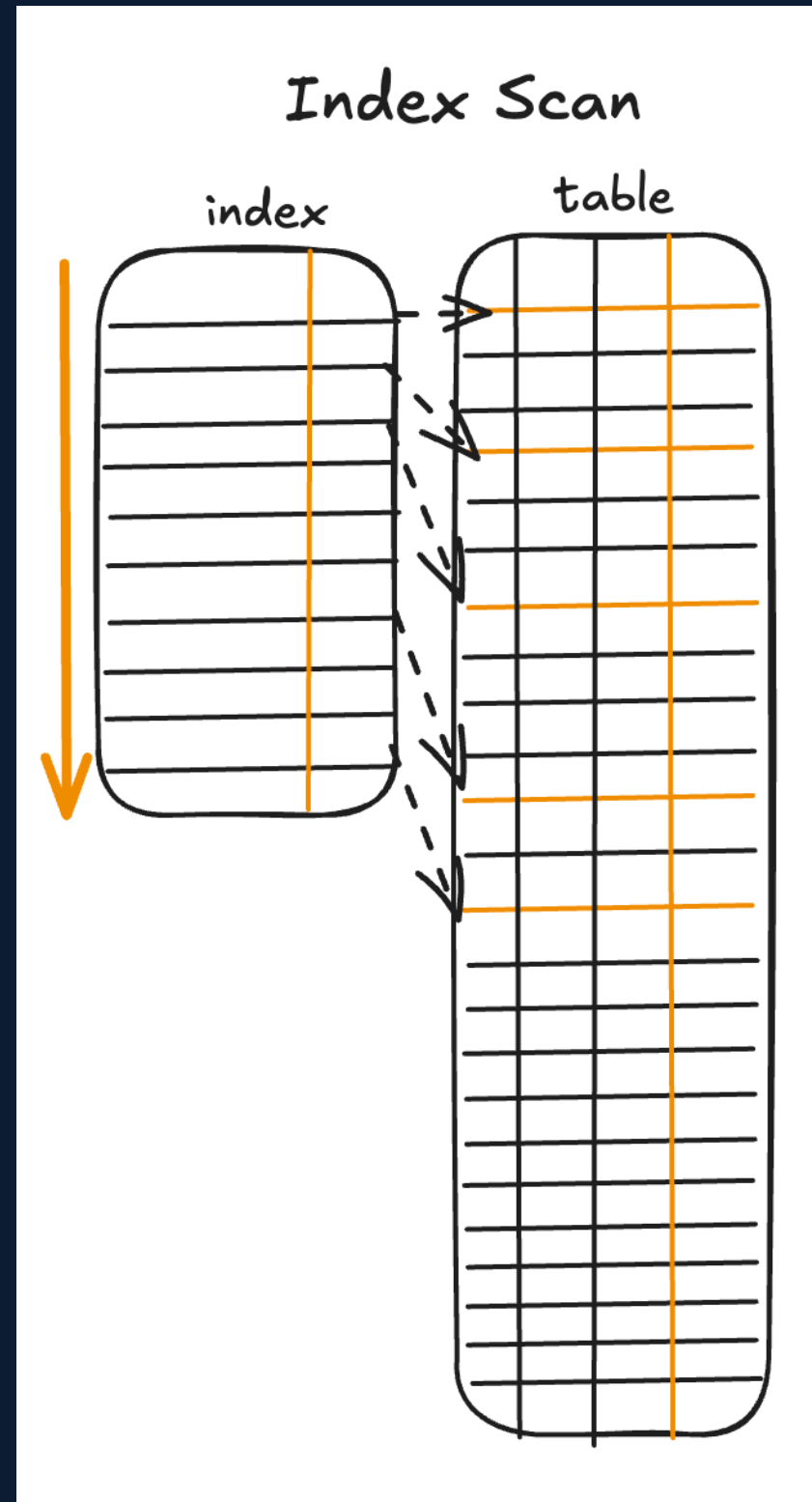
Good when: Small tables, selecting most rows, no useful index



Index Scan

Uses index to find rows, then fetches from table

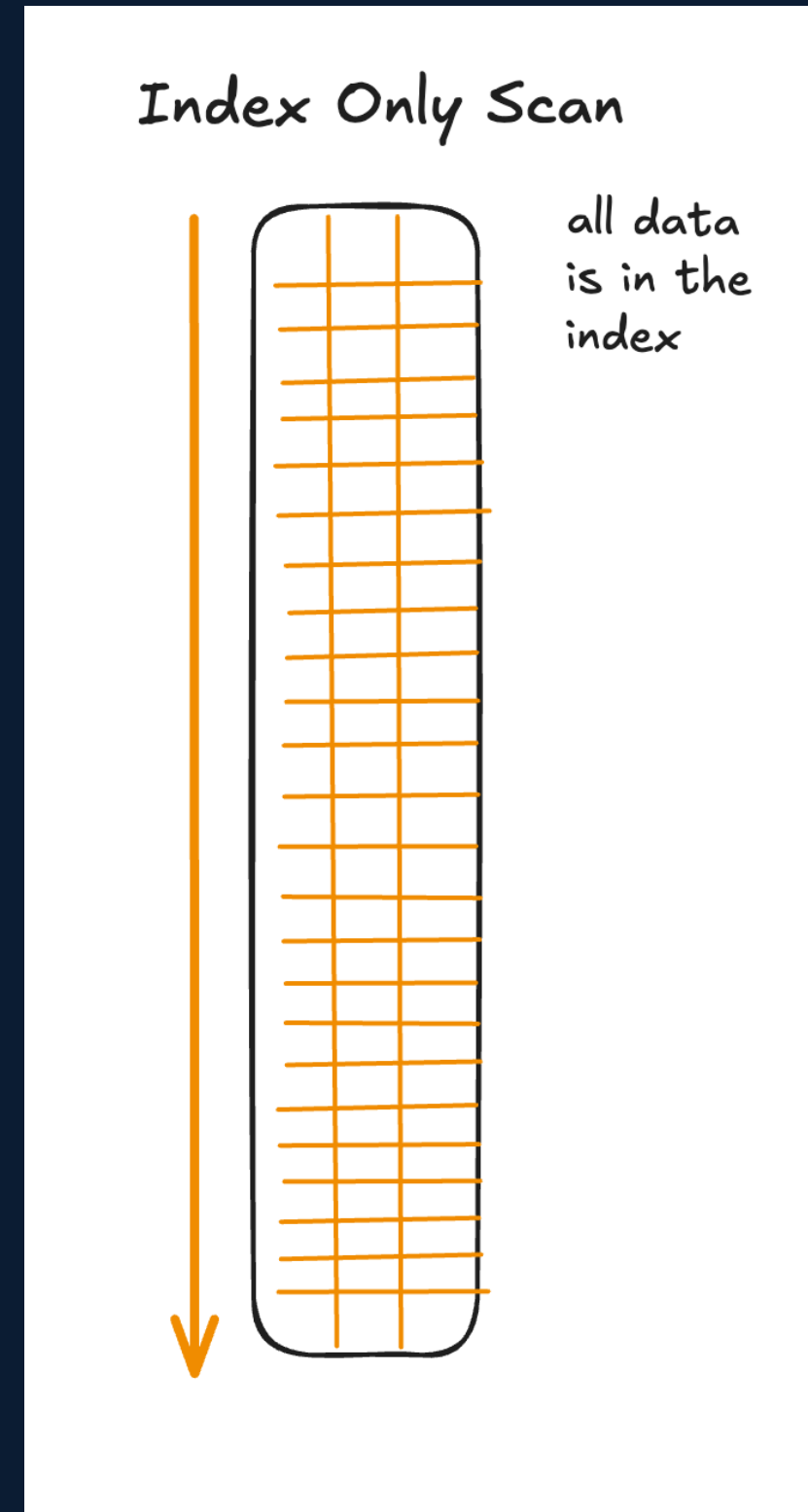
Good when: Selecting small percentage of rows, index matches conditions



Index Only Scan

All needed data is in the index - no table access!

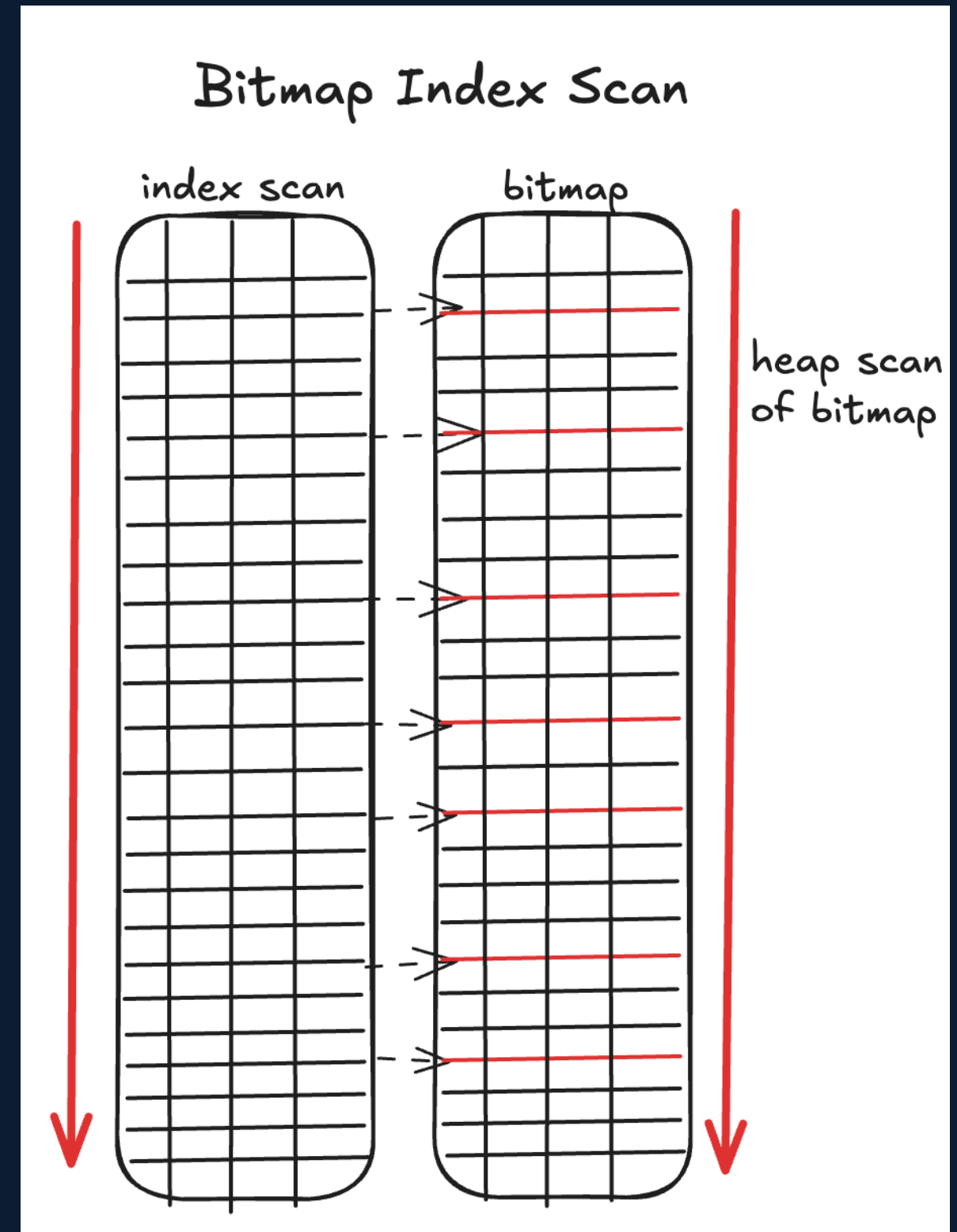
Best case - requires index covering all columns + recent vacuum



Bitmap Scans

Two-phase: Build bitmap of matching rows, then fetch in physical order

Good for: Medium selectivity queries



Join Operations - Nested Loop

```
EXPLAIN SELECT f.title, p.name FROM bluebox.film f
JOIN bluebox.film_cast fc ON f.film_id = fc.film_id
JOIN bluebox.person p ON fc.person_id = p.person_id
WHERE f.film_id = 155;
```

Nested Loop (actual time=0.10..0.16 rows=5)

→ Nested Loop (actual time=0.08..0.08 rows=5)

→ Index Scan on film (film_id = 155)

→ Index Only Scan on film_cast

→ Index Scan on person

Best for small result sets with good indexes!

Sort Methods in EXPLAIN

```
Sort  (cost=1200.00..1250.00 rows=10000)
  Sort Key: vote_average DESC
  Sort Method: quicksort  Memory: 1024kB
```

Or worse:

```
Sort  (cost=1200.00..1250.00 rows=10000)
  Sort Key: vote_average DESC
  Sort Method: external merge  Disk: 10240kB ← BAD!
```

External merge = data exceeded work_mem

Indexes

Important Postgres Indexes

B-tree	Numbers or text that can be ordered or compared Usually with < = > or ORDERBY
Multi-column B-tree	Numbers or text often queried together Also called composite index
Partial B-tree	Numbers or text with low cardinality, often NULLs or frequent value
Covering	Entire result of query is stored in index Often B-tree, but can be other types
Expression	Algebraic expression based on column data and stored with the index Often B-tree, but can be other types
GIN	For composite or multi-value data types like JSONB Also good for wildcards or partial searches with text search
GiST	Geospatial data types with PostGIS May also good for range, nearest neighbor, and some full text search
HNSW	Similarity and neighbor searches with the vector data type
Hash	For text, varchar, integers that are very long like UUIDS or URLs
BRIN	Very large tables, too large for b-tree, often time series data

B-Tree Index (Default)

Best for: Equality, ranges, sorting, LIKE with prefix

-- BEFORE: Check the plan without an index

EXPLAIN ANALYZE

SELECT title FROM bluebox.film WHERE vote_average > 8;

-- Seq Scan on film (cost=0.00..941.95)

-- Filter: (vote_average > 8)

B-Tree Index: After

```
-- Create the index
CREATE INDEX idx_film_vote_avg ON bluebox.film(vote_average);

-- AFTER: Check the plan with the index
EXPLAIN ANALYZE
SELECT title FROM bluebox.film WHERE vote_average > 8;
-- Bitmap Heap Scan on film (cost=5.13..308.25 rows=110)
--   Recheck Cond: (vote_average > 8)
--   -> Bitmap Index Scan on idx_film_vote_avg (cost=0.00..5.11 rows=110)
--       Index Cond: (vote_average > 8)
```

Total cost dropped from ~942 to ~308. Read the top node: the index lookup itself is ~5, fetching the ~100 matching table pages is the rest.

Index Type Summary

Type	Use Case
B-tree	General purpose (default)
GIN	JSONB, arrays, full-text
GiST	Geometry, ranges
Hash	Equality only (rare)
BRIN	Time-series, ordered data

When NOT to Index

- Small tables (seq scan is fine)
- Columns rarely in WHERE/JOIN/ORDER BY
- Low cardinality columns (few distinct values)
- Write-heavy tables with few reads

Every index has maintenance cost!

Common Performance Patterns

Pattern: N+1 Queries

Problem: Loop making individual queries

```
## BAD: N+1 queries
for customer in get_customers():
    rentals = query(f"SELECT * FROM rental WHERE customer_id = {customer.id}")
```

Solution: Single query with JOIN

```
-- GOOD: Single query
SELECT c.*, r.*
FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id;
```

Pattern: SELECT *

Problem: Fetching unnecessary columns

-- BAD: Gets all columns

```
SELECT * FROM bluebox.film WHERE vote_average > 8;
```

Solution: Select only needed columns

-- GOOD: Only needed columns

```
SELECT film_id, title, vote_average  
FROM bluebox.film  
WHERE vote_average > 8;
```

Pattern: OFFSET for Pagination

Problem: OFFSET scans and discards rows

```
-- BAD: Must scan 10000 rows to skip them
SELECT * FROM bluebox.rental
ORDER BY rental_id
OFFSET 10000 LIMIT 20;
```

Solution: Keyset pagination

```
-- GOOD: Use index
SELECT * FROM bluebox.rental
WHERE rental_id > 10000
ORDER BY rental_id
LIMIT 20;
```

Pattern: Functions on Indexed Columns

Problem: Function prevents index use

-- BAD: Can't use index efficiently

```
SELECT * FROM bluebox.payment  
WHERE DATE(payment_date) = '2024-01-15';
```

Solution: Rewrite condition

-- GOOD: Can use index

```
SELECT * FROM bluebox.payment  
WHERE payment_date >= '2024-01-15'  
      AND payment_date < '2024-01-16';
```

Query Tuning Checklist

1.  Check pg_stat_statements for slow queries
2.  Use EXPLAIN (ANALYZE, BUFFERS) to understand plans
3.  Ensure appropriate indexes exist
4.  Look for sequential scans on large tables
5.  Watch for disk sorts (increase work_mem)
6.  Verify statistics are current (ANALYZE)

Session 3 Summary

-  Diagnosing locks and blocking sessions
-  Finding and stopping runaway queries, timeouts as guardrails
-  Monitoring essentials, key metrics and `pg_stat_statements`
-  A few key memory settings: `shared_buffers`, `work_mem`
-  Reading query plans with EXPLAIN
-  Index basics and when (not) to add one
-  Common performance anti-patterns and fixes

This is a diagnostic checklist and a first pass at reading query plans - not deep tuning mastery.

Questions?

That Wraps the Series! Bluebox Stays Up

Keep exploring locks, monitoring, EXPLAIN, and indexes on
your own